# Level 2: Supervised Predictive Modeling (Regression) & Unsupervised Cluster Segmentation

**Phase Objective:** Extract actionable business intelligence by building supervised models to forecast numerical variables and deploying unsupervised learning algorithms to discover latent structures within data features.

## 1. Supervised Learning: Parametric Linear Regression

Linear Regression establishes a deterministic linear relationship between independent input features (X) and a continuous target vector (y, such as Asset or House Prices). The algorithm computes coefficients that minimize the Ordinary Least Squares (OLS) criterion: the sum of squared differences between observed values and predictions.

### Data Partitioning Strategy:

To evaluate predictive generalization and safeguard against overfitting, the dataset is split into independent sub-spaces using random shuffling with stratified controls via seed assignment.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Feature and Target Isolation
X = df.drop('Price', axis=1)
y = df['Price']

# Split dataset with an 80/20 ratio and set seed for reproducibility
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state

# Model Training
reg_model = LinearRegression()
reg_model.fit(X_train, y_train)
```

```
# inference
y_pred = reg_model.predict(X_test)
```

### Mathematical Performance Evaluation:

Evaluation requires two metrics: Mean Squared Error (MSE) and the Coefficient of Determination ($R^2$ Score).

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error: {mse:.4f}")
print(f"R2 Coefficient of Determination: {r2:.4f}")
```

**Metric Interpretation:** MSE quantifies model variance by averaging squared prediction residuals, penalizing larger deviations heavily. The $R^2$ score measures goodness-of-fit, bounded between $-\infty$ and $1.0$. An $R^2$ of $0.85$ signifies that $85\%$ of the total variance in the target variable is successfully explained by the model's features.

## 2. Unsupervised Learning: Spatial K-Means Clustering

K-Means separates unlabeled multidimensional points into distinct geometric partitions based on spatial distance proximity. It iteratively updates centroid coordinates to minimize the within-cluster sum of squares (inertia).

### The Critical Role of Feature Scaling:

Because K-Means depends on Euclidean distance measures, features with large absolute numerical scales (e.g., Annual Income) dominate features with smaller scales (e.g., Age). This shifts distance calculations and distorts cluster shapes. Standardizing features resolves this scaling bias.

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import seaborn as sns
import matplotlib.pyplot as plt

# Isolate features for spatial clustering
features_to_cluster = df[['FeatureA', 'FeatureB']]

# Apply Z-score transformation: Mean=0, Std=1
```

```
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features_to_cluster)

# Initialize and fit K-Means with 3 target centroids
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster_Labels'] = kmeans.fit_predict(scaled_features)
```

**Technical Breakdown:** `StandardScaler` applies a Z-score transformation: $z = (x - \mu) / \sigma$. This centers data at zero and establishes a unit standard deviation. `n_init=10` configures the model to run 10 independent centroid initializations via the K-Means++ algorithm, picking the run with the lowest overall inertia to avoid poor local minima.

### Statistical Visualizations:

Two essential charts are built to support the analysis: a Correlation Heatmap and a Cluster Map.

```
# 1. Linear Multi-collinearity Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Matrix Heatmap")

# 2. Cluster Spatial Plot
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='FeatureA', y='FeatureB', hue='Cluster_Labels', palette='S
plt.title("K-Means Cluster Infiltration Map (k=3)")
plt.show()
```